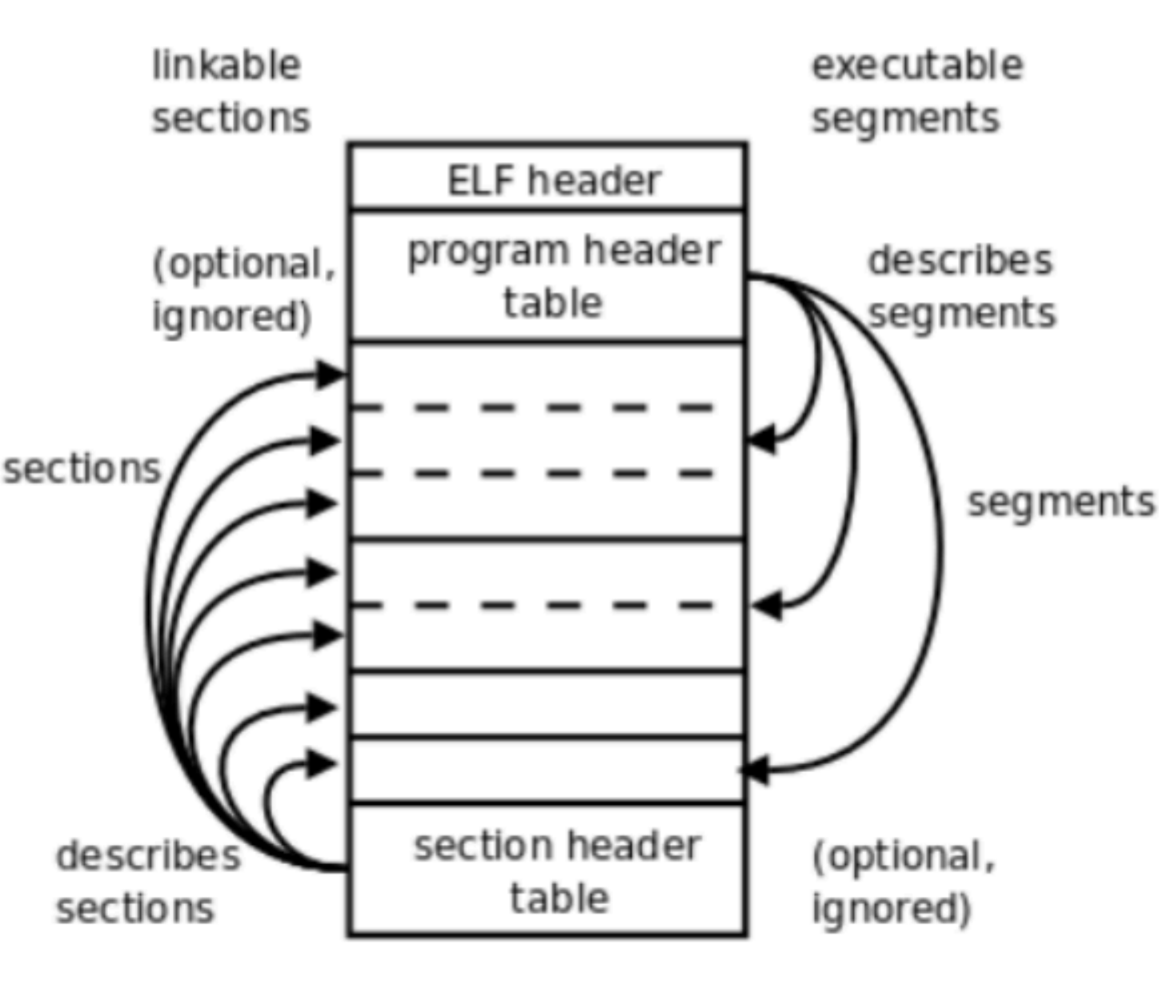
tmp0ut首刊第二篇文章知识点拆解

参考文章：`https://tmpout.sh/1/2.html`

一.ELF文件简述

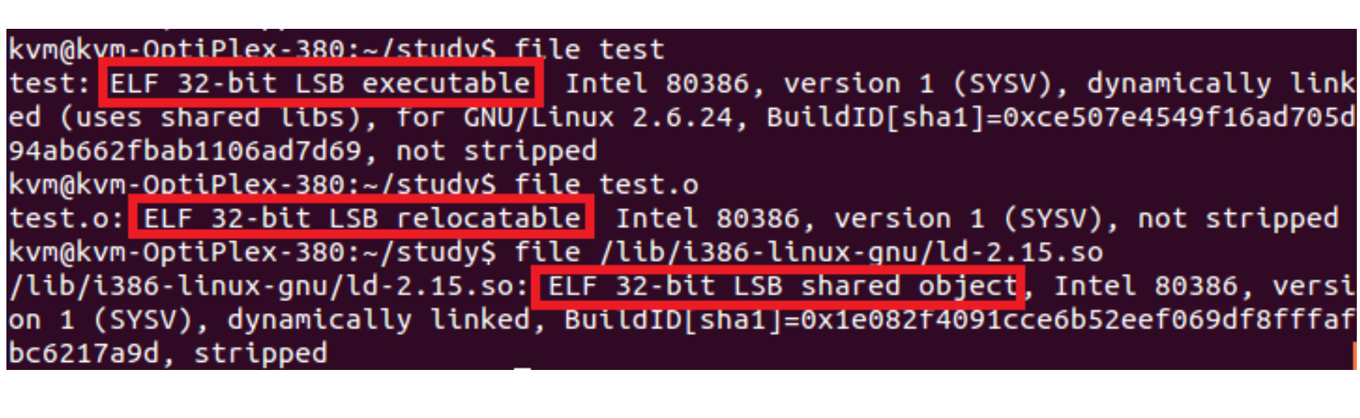


> 上图是elf程序的构成，左边是构建视图（即程序构建的时候需要的），右边是执行视图（即程序执行时需要的）
>
> 程序构建时，需要elf头，以及节头表中描述的信息，即左侧箭头指向的，根据节头表去合并各个“节”，此时忽略程序头表。
>
> 程序执行时，则不需要节头表了，只需要elf头，程序头中描述的段信息，根据程序头表把多个“节”合并成的“段（Segment）”加载到内存中，忽略节头表。节头表主要提供给readelf等分析器使用。

elf的类型

ELF主要包括三种类型文件：

- 可重定位文件(relocatable)：编译器和汇编器产生的.o文件，被Linker所处理
- 可执行文件(executable)：Linker对.o文件进行处理输出的文件，进程映像
- 共享对象文件(shared object)：动态库文件.so



```
kvm@kvm-OptiPlex-380:~/study$ file test
test: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked (uses shared libs), for GNU/Linux 2.6.24, BuildID[sha1]=0xce507e4549f16ad705d94ab662fbab1106ad7d69, not stripped
kvm@kvm-OptiPlex-380:~/study$ file test.o
test.o: ELF 32-bit LSB relocatable, Intel 80386, version 1 (SYSV), not stripped
kvm@kvm-OptiPlex-380:~/study$ file /lib/i386-linux-gnu/ld-2.15.so
/lib/i386-linux-gnu/ld-2.15.so: ELF 32-bit LSB shared object, Intel 80386, version 1 (SYSV), dynamically linked, BuildID[sha1]=0x1e082f4091cce6b52eef069df8fffafbc6217a9d, stripped
```

elf头和程序头

在Linux源码中，这二者主要表现为这两个结构体，具体路径为

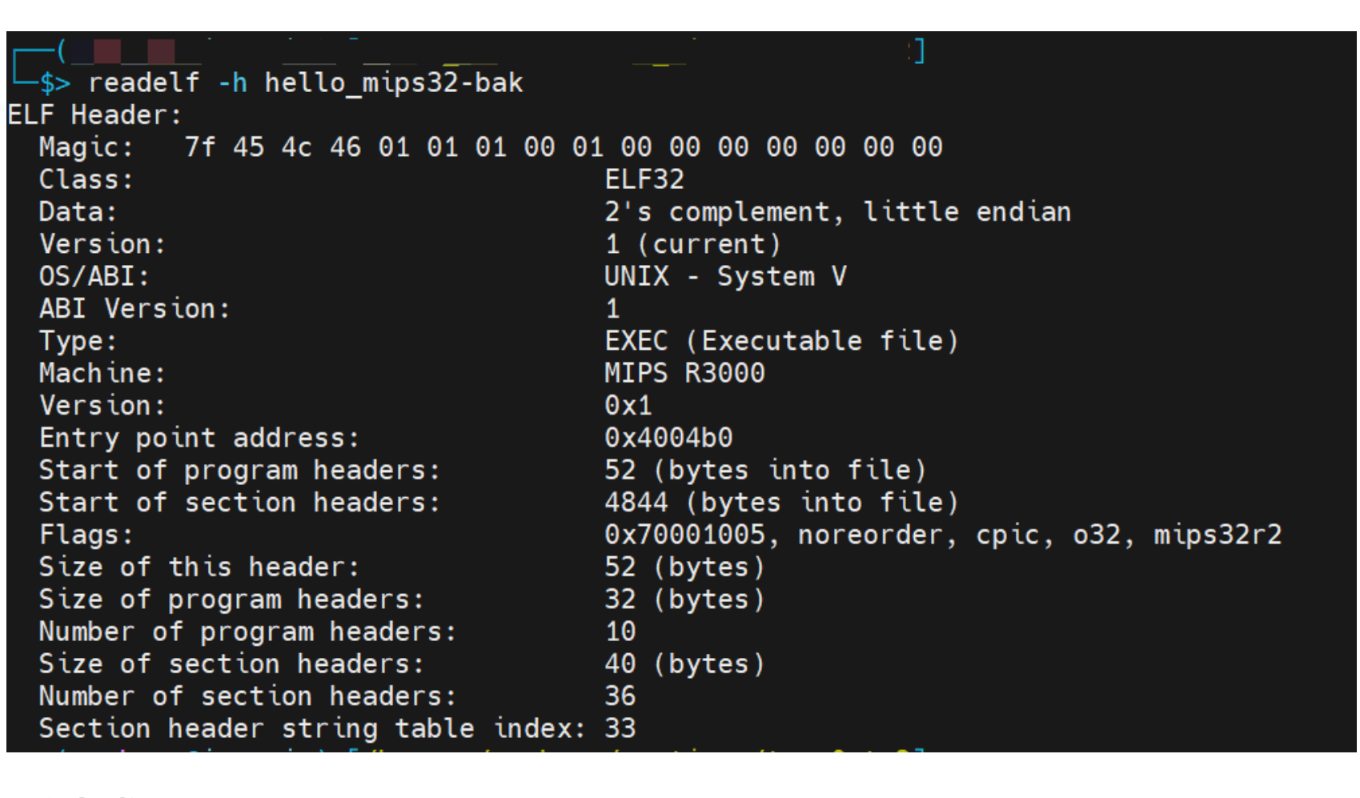
`kernel-4.4.94/include/uapi/linux/elf.h`

其中，elf头如下

```
#define EI_NIDENT 16
typedef struct{
    /*ELF的一些标识信息，固定值*/
    unsigned char e_ident[EI_NIDENT];
    /*目标文件类型：1-可重定位文件，2-可执行文件，3-共享目标文件等*/
    Elf32_Half e_type;
    /*文件的目标体系结构类型：3-intel 80386*/
    Elf32_Half e_machine;
    /*目标文件版本：1-当前版本*/
    Elf32_Word e_version;
    /*程序入口的虚拟地址，如果没有入口，可为0*/
    Elf32_Addr e_entry;
    /*程序头表(segment header table)的偏移量，如果没有，可为0*/
    Elf32_Off e_phoff;
    /*节区头表(section header table)的偏移量，没有可为0*/
    Elf32_Off e_shoff;
    /*与文件相关的，特定于处理器的标志*/
    Elf32_Word e_flags;
    /*ELF头部的大小，单位字节*/
    Elf32_Half e_ehsize;
    /*程序头表每个表项的大小，单位字节*/
    Elf32_Half e_phentsize;
    /*程序头表表项的个数*/
    Elf32_Half e_phnum;
    /*节区头表每个表项的大小，单位字节*/
    Elf32_Half e_shentsize;
    /*节区头表表项的数目*/
    Elf32_Half e_shnum;
    /*某些节区中包含固定大小的项目，如符号表。对于这类节区，此成员给出每个表项的长度字节数。*/
    Elf32_Half e_shstrndx;
}Elf32_Ehdr;
```

> 注意，上面的e_shstrndx需要额外讲一下，这是指向节头表（节头表是一个数组）某一位置的索引。该索引对应的节包含节头表中每一节的名称

示例



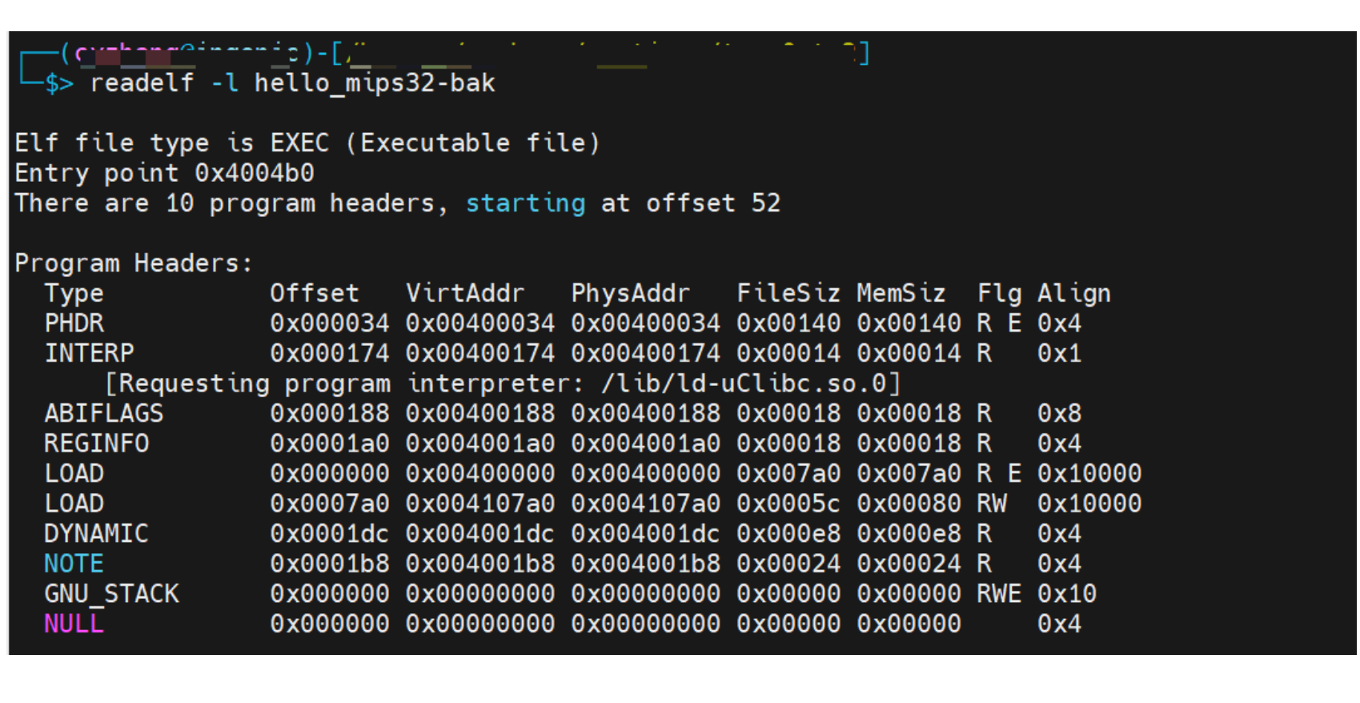
```
┌──([redacted])-[[redacted]]
└─$> readelf -h hello_mips32-bak
ELF Header:
  Magic:   7f 45 4c 46 01 01 01 00 01 00 00 00 00 00 00 00
  Class:                             ELF32
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       1
  Type:                              EXEC (Executable file)
  Machine:                           MIPS R3000
  Version:                           0x1
  Entry point address:               0x4004b0
  Start of program headers:          52 (bytes into file)
  Start of section headers:          4844 (bytes into file)
  Flags:                             0x70001005, noreorder, cpic, o32, mips32r2
  Size of this header:               52 (bytes)
  Size of program headers:           32 (bytes)
  Number of program headers:         10
  Size of section headers:           40 (bytes)
  Number of section headers:         36
  Section header string table index: 33
```

程序头则如下

> 程序头部分，不只是一份这个结构体，而是以一个结构体数组的形式存在，该数组的每个成员，都是下面的一个结构体
>
> ，因此严格来说，应该是程序头表，表中的每一项是程序头，不过一般都直接说程序头
>
> 示例

```
typedef struct
{
    /*segment的类型：PT_LOAD= 1 可加载的段*/
    Elf32_Word p_type;
    /*从文件头到该段第一个字节的偏移*/
    Elf32_Off p_offset;
    /*该段第一个字节被放到内存中的虚拟地址*/
    Elf32_Addr p_vaddr;
    /*在linux中这个成员没有任何意义，值与p_vaddr相同*/
    Elf32_Addr p_paddr;
    /*该段在文件映像中所占的字节数*/
    Elf32_Word p_filesz;
    /*该段在内存映像中占用的字节数*/
    Elf32_Word p_memsz;
    /*段标志*/
    Elf32_Word p_flags;
    /*p_vaddr是否对齐*/
    Elf32_Word p_align;
} Elf32_phdr;

//在32位mips中，其内存分布为
        ├─ p_type    4 字节
        ├─ p_offset  4 字节
        ├─ p_vaddr   4 字节
        ├─ p_paddr   4 字节
        ├─ p_filesz  4 字节
        ├─ p_memsz   4 字节
        ├─ p_flags   4 字节
        └─ p_align   4 字节
```

示例

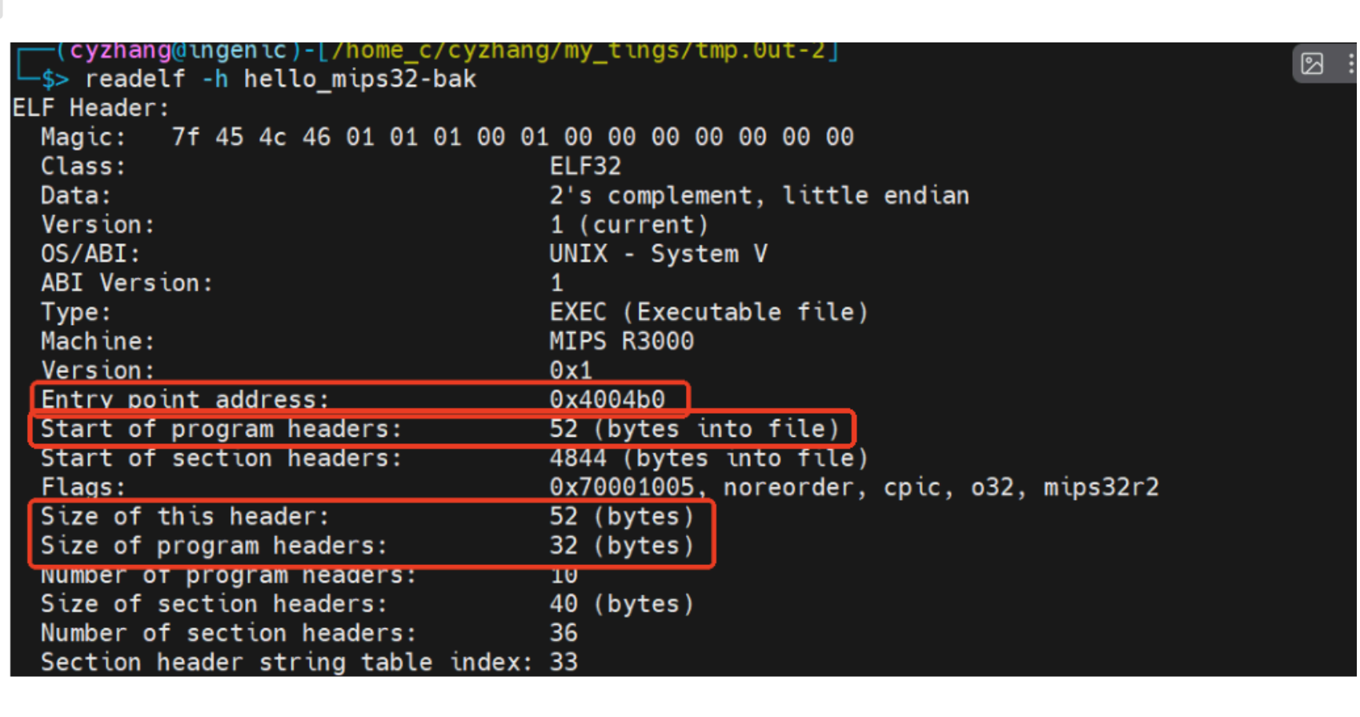


```
┌──([redacted])-[[redacted]]
└─$> readelf -l hello_mips32-bak

Elf file type is EXEC (Executable file)
Entry point 0x4004b0
There are 10 program headers, starting at offset 52

Program Headers:
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  PHDR           0x000034 0x00400034 0x00400034 0x00140 0x00140 R E 0x4
  INTERP         0x000174 0x00400174 0x00400174 0x00014 0x00014 R   0x1
      [Requesting program interpreter: /lib/ld-uClibc.so.0]
  ABIFLAGS       0x000188 0x00400188 0x00400188 0x00018 0x00018 R   0x8
  REGINFO        0x0001a0 0x004001a0 0x004001a0 0x00018 0x00018 R   0x4
  LOAD           0x000000 0x00400000 0x00400000 0x007a0 0x007a0 R E 0x10000
  LOAD           0x0007a0 0x004107a0 0x004107a0 0x0005c 0x00080 RW  0x10000
  DYNAMIC        0x0001dc 0x004001dc 0x004001dc 0x000e8 0x000e8 R   0x4
  NOTE           0x0001b8 0x004001b8 0x004001b8 0x00024 0x00024 R   0x4
  GNU_STACK      0x000000 0x00000000 0x00000000 0x00000 0x00000 RWE 0x10
  NULL           0x000000 0x00000000 0x00000000 0x00000 0x00000     0x4
```

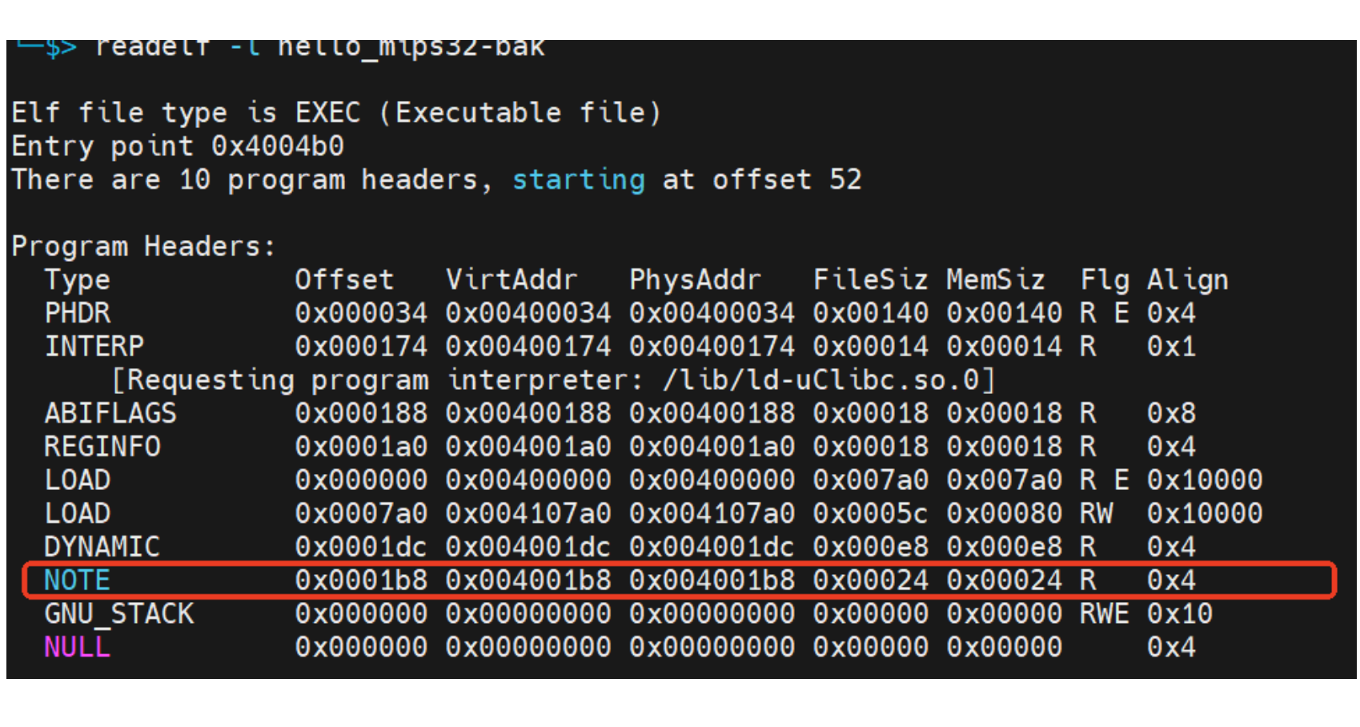
二.ELF注入步骤

1.拿到目标elf文件，使用解析器解析其elf头和程序

> 红框中的，是需要关注的信息



```
┌──(cyzhang@ingenic)-[/home_c/cyzhang/my_tings/tmp.0ut-2]
└─$> readelf -h hello_mips32-bak
ELF Header:
  Magic:   7f 45 4c 46 01 01 01 00 01 00 00 00 00 00 00 00
  Class:                             ELF32
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       1
  Type:                              EXEC (Executable file)
  Machine:                           MIPS R3000
  Version:                           0x1
  Entry point address:               0x4004b0
  Start of program headers:          52 (bytes into file)
  Start of section headers:          4844 (bytes into file)
  Flags:                             0x70001005, noreorder, cpic, o32, mips32r2
  Size of this header:               52 (bytes)
  Size of program headers:           32 (bytes)
  Number of program headers:         10
  Size of section headers:           40 (bytes)
  Number of section headers:         36
  Section header string table index: 33
```



```
└─$> readelf -l hello_mips32-bak

Elf file type is EXEC (Executable file)
Entry point 0x4004b0
There are 10 program headers, starting at offset 52

Program Headers:
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  PHDR           0x000034 0x00400034 0x00400034 0x00140 0x00140 R E 0x4
  INTERP         0x000174 0x00400174 0x00400174 0x00014 0x00014 R   0x1
      [Requesting program interpreter: /lib/ld-uClibc.so.0]
  ABIFLAGS       0x000188 0x00400188 0x00400188 0x00018 0x00018 R   0x8
  REGINFO        0x0001a0 0x004001a0 0x004001a0 0x00018 0x00018 R   0x4
  LOAD           0x000000 0x00400000 0x00400000 0x007a0 0x007a0 R E 0x10000
  LOAD           0x0007a0 0x004107a0 0x004107a0 0x0005c 0x00080 RW  0x10000
  DYNAMIC        0x0001dc 0x004001dc 0x004001dc 0x000e8 0x000e8 R   0x4
  NOTE           0x0001b8 0x004001b8 0x004001b8 0x00024 0x00024 R   0x4
  GNU_STACK      0x000000 0x00000000 0x00000000 0x00000 0x00000 RWE 0x10
  NULL           0x000000 0x00000000 0x00000000 0x00000 0x00000     0x4
```

2.构建裸二进制文件，即要注入的文件

> 建议使用汇编构造，如果使用c的话，可能存在依赖问题（需要依赖uclibc什么的），而且不利于控制生成物大小，对汇编不熟的可以使用AI辅助构建，很简单的。不过在不同架构中，需要尽量规避可能影响生成物大小的点，以精确控制生成物大小。

示例以及详解

```
//此代码为MIPS架构
//二进制文件源码
.set noreorder //关闭自动重排，MIPS存在分支延迟槽，若不关闭这个，则汇编器会在跳转指令后自动填充，从而增大生成物大小
.set noat //禁止使用at临时寄存器，防止其进行命令展开，从而增大生成物大小
.text  //声明后续属于可执行代码段，方便后续提取
.globl _start

_start:
li $v0, 4004
li $a0, 1
lui $a1, %hi(msg)
addiu $a1,$a1,%lo(msg)
li $a2, msg_len
syscall

lui $t9,%hi(0x004004b0)
addiu $t9,$t9, %lo(0x004004b0)
jr $t9 //跳转到最开始的elf加载入口，即前面所标出的Entry point address
nop

msg:
   .ascii "hi hi hi,this is made by zcy!\n"
msg_len = . - msg
```

2-1.将上述代码编译后，使用这个指令剥离其余部分，只留下裸二进制执行程序部分

```
mips-linux-gnu-objcopy -O binary -j .text payload.elf payload.bin
//不同架构具体命令可能不一样，但是基本都是使用objcopy
```

> 这个命令的原理是提取elf文件中的.text段，这个信息来源于节头表，数据存储在内容区，不懂的可以返回文档顶看一下哪个图

```
        文件格式：ELF
         │
         ├─ ELF 头                            64 字节
         │
         ├─ 程序头表                   13 × 56 = 728 字节
         │    └─ 第 4 条 LOAD (R E) : 0x4000 ~ 0x17581
         │          └─ 把 .text 那 75,730 字节包进可执行段
         │
         ├─ 内容区                        139,409 字节
         │    ├─ .interp
         │    ├─ .note.*
         │    ├─ .gnu.hash / .dynsym / .dynstr
         │    ├─ .rela.dyn / .rela.plt
         │    ├─ .init / .plt / .plt.sec
         │    ├─ .text      0x4DA0 起，75,730 字节   ←── 就是它
         │    ├─ .fini
         │    ├─ .rodata / .eh_frame
         │    ├─ .init_array / .dynamic / .got
         │    ├─ .data / .bss
         │    └─ … (共 30 个节)
         │
         └─ 节头表                     30 × 64 = 1,920 字节
              └─ 第 16 条记录:
                   sh_name  = 0xA9  →  ".text"
                   sh_flags = A | X
                   sh_addr  = 0x4DA0
                   sh_size  = 0x127D2
```

2-2 将上述文件连接到目标elf尾部

```
cat elf_practice/payload.bin >> hello_mips32
```

> 更进阶的方式是直接塞入elf文件中由于填充和对齐产生的padding中，这样可以在不改变文件大小的前提下完成注入，但此处不予演示了

2-3修改P_type

```
printf '\x01\x00\x00\x00' | dd of=hello_mips32 bs=1 seek=$((0x114)) conv=notrunc
//把PT_NOTE段的p_type修改为PT_LOAD，这样就可以用来执行了
```

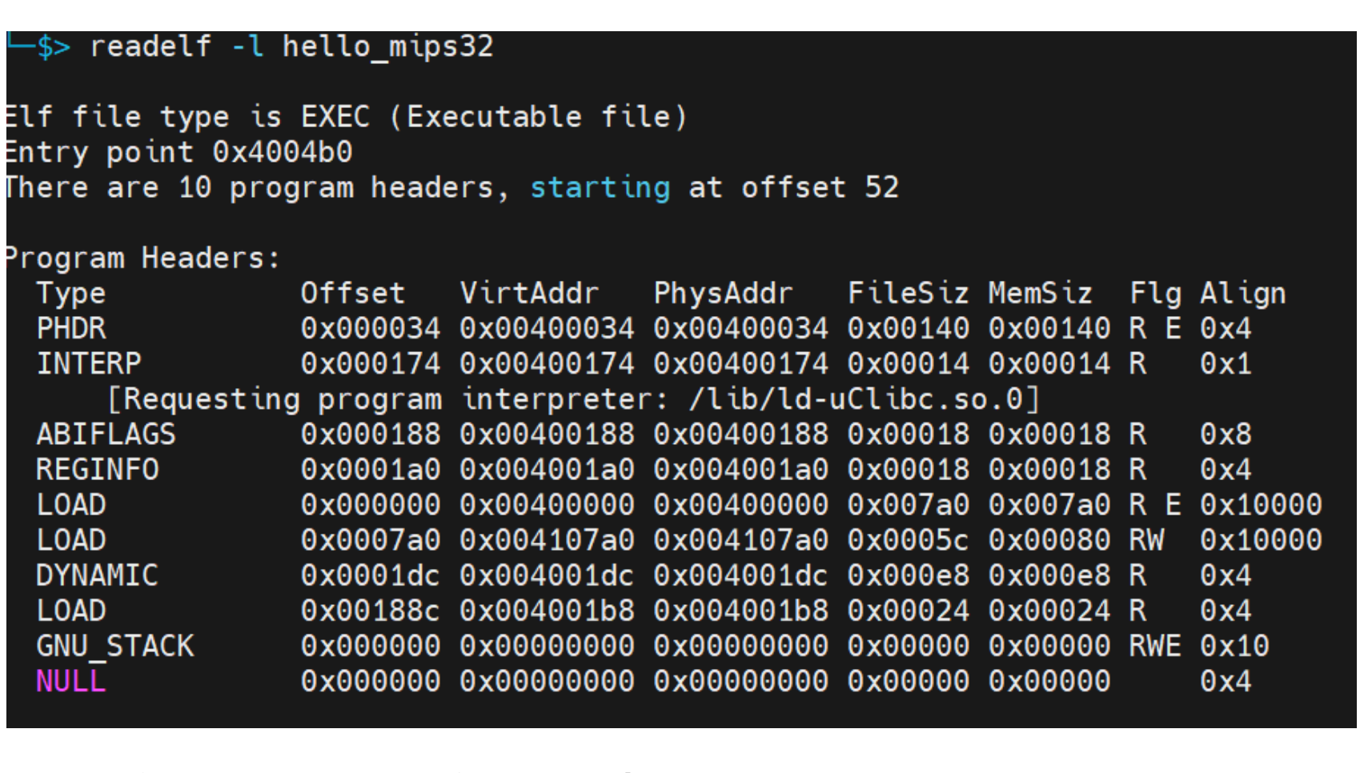
> 上述语句的含义是，将\x01\x00\x00\x00四个字节，直接写入从文件（hello_mips32，即目标文件）开头处偏移0x114处，每次写一字节，conv=notrunc的含义是不截断之后的数据，这个必须要加，否则写完之后文件会废掉。

> 为什么是0x114?
>
> 52+32*7=276，其16进制就是0x114，具体我就不展开了，可以看前面的ELF头部分标红框的哪几个信息。

> 为什没修改PT_NOTE这一段?
>
> 因为绝大部分elf中，这一段在真正运行的时候不起作用，若修改其他段，可能导致原elf无法运行

> 此处我这么组织这四个字节，是因为MIPS架构的小端序，其他架构应该不可以直接复制，可让ai适配一下

修改后，再用readelf读取，即可发现其已经变为LOAD，修改成功

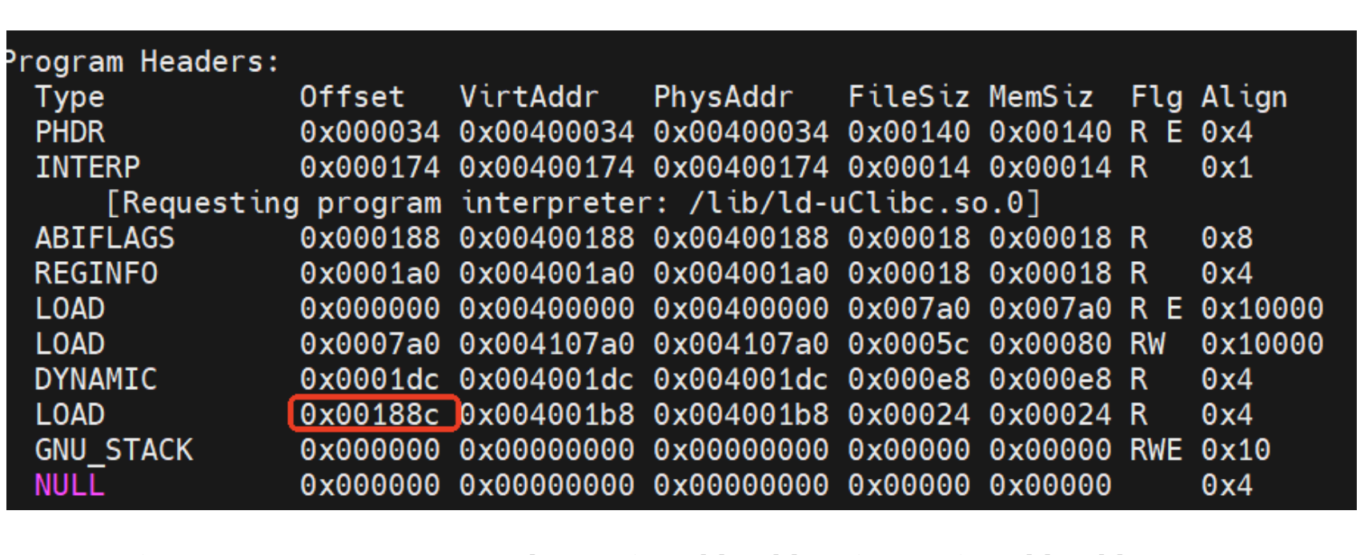


```
└─$> readelf -l hello_mips32

Elf file type is EXEC (Executable file)
Entry point 0x4004b0
There are 10 program headers, starting at offset 52

Program Headers:
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  PHDR           0x000034 0x00400034 0x00400034 0x00140 0x00140 R E 0x4
  INTERP         0x000174 0x00400174 0x00400174 0x00014 0x00014 R   0x1
      [Requesting program interpreter: /lib/ld-uClibc.so.0]
  ABIFLAGS       0x000188 0x00400188 0x00400188 0x00018 0x00018 R   0x8
  REGINFO        0x0001a0 0x004001a0 0x004001a0 0x00018 0x00018 R   0x4
  LOAD           0x000000 0x00400000 0x00400000 0x007a0 0x007a0 R E 0x10000
  LOAD           0x0007a0 0x004107a0 0x004107a0 0x0005c 0x00080 RW  0x10000
  DYNAMIC        0x0001dc 0x004001dc 0x004001dc 0x000e8 0x000e8 R   0x4
  LOAD           0x00188c 0x004001b8 0x004001b8 0x00024 0x00024 R   0x4
  GNU_STACK      0x000000 0x00000000 0x00000000 0x00000 0x00000 RWE 0x10
  NULL           0x000000 0x00000000 0x00000000 0x00000 0x00000     0x4
```

2-4：修改p_offset，使其指向payload起点

```
printf '\x8C\x18\x00\x00' | dd of=hello_mips32 bs=1 seek=$((0x118)) conv=notrunc
//输入的字节为0x188c，即文件大小的16进制，也即payload的起点
```

修改完可以发现



```
Program Headers:
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  PHDR           0x000034 0x00400034 0x00400034 0x00140 0x00140 R E 0x4
  INTERP         0x000174 0x00400174 0x00400174 0x00014 0x00014 R   0x1
      [Requesting program interpreter: /lib/ld-uClibc.so.0]
  ABIFLAGS       0x000188 0x00400188 0x00400188 0x00018 0x00018 R   0x8
  REGINFO        0x0001a0 0x004001a0 0x004001a0 0x00018 0x00018 R   0x4
  LOAD           0x000000 0x00400000 0x00400000 0x007a0 0x007a0 R E 0x10000
  LOAD           0x0007a0 0x004107a0 0x004107a0 0x0005c 0x00080 RW  0x10000
  DYNAMIC        0x0001dc 0x004001dc 0x004001dc 0x000e8 0x000e8 R   0x4
  LOAD           0x00188c 0x004001b8 0x004001b8 0x00024 0x00024 R   0x4
  GNU_STACK      0x000000 0x00000000 0x00000000 0x00000 0x00000 RWE 0x10
  NULL           0x000000 0x00000000 0x00000000 0x00000 0x00000     0x4
```

2-5：修改p_vaddr和p_paddr（虚拟内存地址基址和物理内存地址基址）

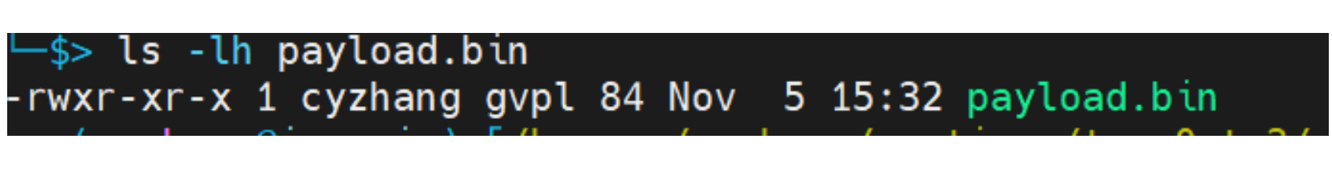
```
printf '\x8C\x18\x00\x0C' | dd of=hello_mips32 bs=1 seek=$((0x11C)) conv=notrunc
printf '\x8C\x18\x00\x0C' | dd of=hello_mips32 bs=1 seek=$((0x120)) conv=notrunc
```

> 为什么是0x11c和0x120?
>
> 在前面计算好的0x114（这个也就是当前修改的这个程序头）的基础上，根据结构体中每个成员的数据类型大小，递推。由于该结构体中所有成员都是4字节，是对齐的，因此无需考虑结构体的对齐。
>
> **PS:什么是结构体的对齐，可以看附录1，我有讲**
>
> 为什么是0x0c00188c?
>
> 0c000000处基本没人用，因此将payload的放到此处执行，不会损坏其他程序，从而导致报错。
>
> 而加上188c（即offset），是因为内核计算这一段时，会通过下述方式计算出页边界，完成页取整的操作，再同mmap完成虚拟地址与物理地址之间的转换。若基址选择0c000000，则因为其页取整后偏移为0，而文件页取整后偏移为0x88c，而违背了一一对应的原则
>
> ```
> 页内偏移 = p_vaddr & 0xFFF = 0x0C00188C & 0xFFF = 0x88C
> //0xFFF=4095，而4096正好是一页的大小，页则是内存操作的最小单位
>
> 映射的内存起始 = p_vaddr - 页内偏移 = 0x0C00188C - 0x88C = 0x0C001000
> 映射的文件偏移 = p_offset - 页内偏移 = 0x188C - 0x88C = 0x1000
> 映射长度      = p_filesz + 页内偏移 = 0x54   + 0x88C = 0x8E0
> //为什么要这么计算，因为页是最小的操作单位，因此必须整页复制，而又因为为了维持物理内存和虚拟内存的一一对应，因此直接使用
> p_vaddr & 0xFFF 作为页内偏移进行运算
> //于是内核执行下面这个
> mmap(地址 = 0x0C001000, 长度 = 0x8E0, 权限 = 读+执行, 文件偏移 = 0x1000)
> ```

2-6：修改 `p_filesz` 和 `p_memsz` （程序段待执行内容的大小，和其在mem上所需要占用的大小）

```
printf '\x54\x00\x00\x00' | dd of=hello_mips32 bs=1 seek=$((0x124)) conv=notrunc
printf '\x54\x00\x00\x00' | dd of=hello_mips32 bs=1 seek=$((0x128)) conv=notrunc
```

> PS:此处的标准是 `p_memsz` 必须大于 `p_filesz` ，此处采取的值是相等，因为使用汇编构建，因此不需要冗余
>
> 正常情况下，二者之差会被填充为0，同时作为C 语言里**未初始化的全局变量**（ `.bss` 节）的存放位置

> 0x54是84，即payload.bin的大小
>
> 
>
> ```
> └─$> ls -lh payload.bin
> -rwxr-xr-x 1 cyzhang gvpl 84 Nov  5 15:32 payload.bin
> ```

2-7：修改 `p_flags` （添加执行权限）

```
printf '\x05\x00\x00\x00' | dd of=hello_mips32 bs=1 seek=$((0x12C)) conv=notrunc
//5 = PF_R(4) | PF_X(1)，表示"可读 + 可执行"。原来的值是 4（只有可读）
```

2-8:改 `p_align`

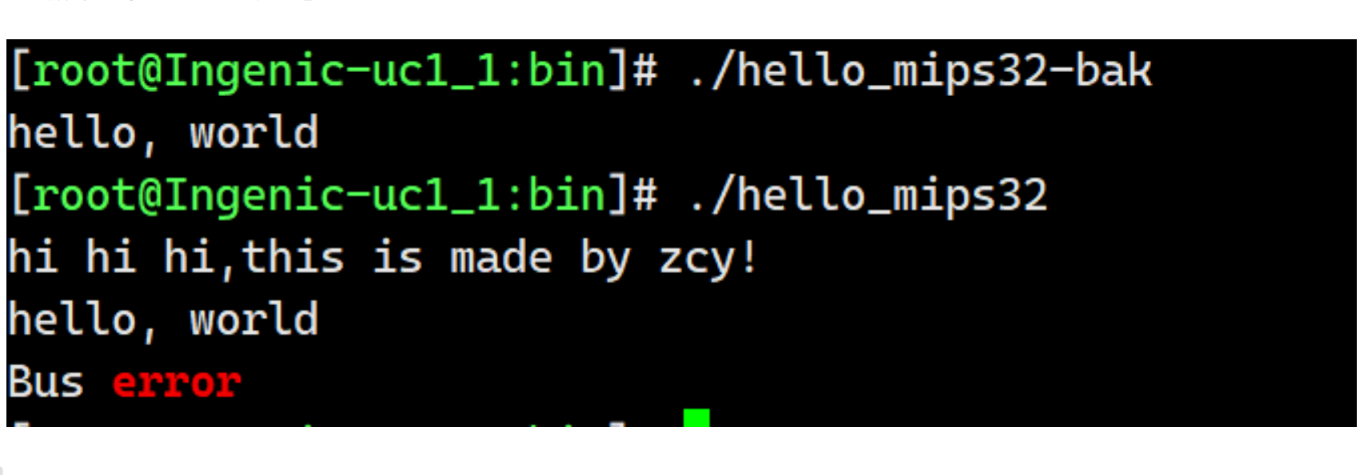
```
printf '\x00\x10\x00\x00' | dd of=hello_mips32 bs=1 seek=$((0x130)) conv=notrunc
//0x1000 = 4096，页大小。这告诉内核这个段需要按 4KB 对齐。
```

> 为什么要修改这个?
>
> 因为内存的最小操作单位是页，我们前面有mmap操作，需要按照这个这个标准对齐
>
> 为什么原来作为NOTE段时，是按照字节对齐的?
>
> NOTE段因为其在节头表中含有A标志，因此被前面的LOAD一起带入内存。而我们修改之后，该段变成了新的LOAD，因此需要更改

2-9：改 `e_entry`

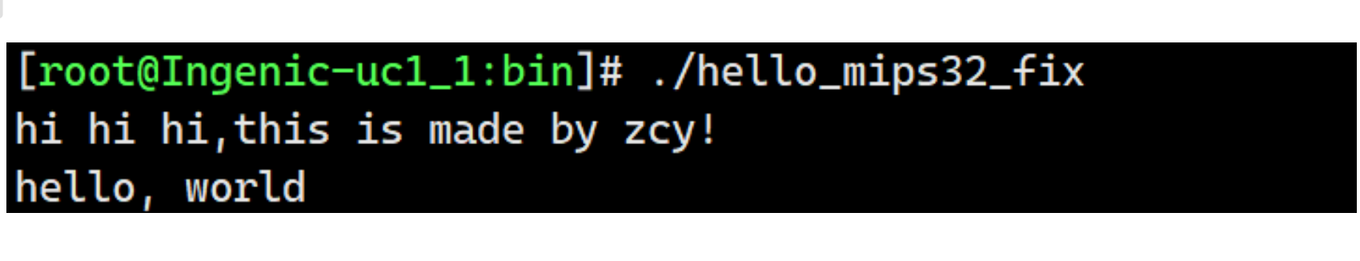
```
printf '\x90\x18\x00\x0C' | dd of=hello_mips32 bs=1 seek=$((0x18)) conv=notrunc
//目的是让elf先去执行我们的代码，然后再执行它自己的
```

3.板端运行现象



```
[root@Ingenic-uc1_1:bin]# ./hello_mips32-bak
hello, world
[root@Ingenic-uc1_1:bin]# ./hello_mips32
hi hi hi,this is made by zcy!
hello, world
Bus error
```

> 上述分别为原版，和注入版的修改现象。可以看到，程序确实先去执行了我们的注入程序，再执行了其自己自带的程序。
>
> 不过后面还有bus error报错，这个问题是由于注入的哪段汇编程序中，不小心将值覆盖进了链接器要使用的一个寄存器v0，因此产生此报错。修复后正常（由于我对MIPS汇编不熟，因此涉及到汇编，都是使用AI辅助写的）



```
[root@Ingenic-uc1_1:bin]# ./hello_mips32_fix
hi hi hi,this is made by zcy!
hello, world
```

附录：

1.C语言结构体的大小，对齐，以及偏移——以及padding泄露

c语言结构体的偏移计算方式

C 语言结构体成员的偏移量，是从 0 开始，根据上一个成员结束的位置，按当前成员自身的对齐值向上取整得到的；而结构体的总大小，则必须按所有成员中最大的对齐值向上取整补齐

> 此处提到的对齐值，是根据ABI，编译器，以及编译选项和配置，规定好的

举例说明

```
struct demo { char a; int b; char c; };

//char 的大小是1字节，int的则是4字节
/*
偏移依次为
a:0
b:4  因为1不为4的倍数，所以添加到4
c:8  b起始为4，自身大小为4，因此c起始为8
*/
```

c语言结构体的大小计算方式和对齐方式

在大小上，C语言结构体的最终大小，是在偏移的计算结果上，基于最大的成员类型的大小，进行补齐，使总大小为最大的成员大小的倍数。

举例说明

```
struct demo { char a; int b; char c; };

//char 的大小是1字节，int的则是4字节

/*
根据上面偏移所计算，三个成员总大小占据了9字节
但是9并非最大的成员（也就是int）的大小的倍数，因此填补到12。因此该结构体大小为12字节

*/
```

padding泄露

可以注意到，上述过程中，存在很多空洞，理论上在ABI，编译器，编译配置，以及内核配置等一系列环境对齐的情况下，这些空洞是可预测的。

而当用户使用结构体时，没有进行初始化，该结构体的空洞中留存的是栈上之前的数据，而此时将结构体整体拷贝，则可以将该数据带出。

```
//上述提到的初始化，是指下面的两种，为结构体的每一个成员赋值，不算此处提到的初始化。因为空洞的值你赋不进去哈哈
struct demo d = {0};
// 或者
struct demo d;
memset(&d, 0, sizeof(d));
```

但是更常用的，是利用用户未初始化，且未赋值的结构体成员，这些往往更好预测，不过原理与这个一致